

# P0516r0

## Clarify That `shared_future`'s Copy Operations have Wide Contracts

Published Proposal, 10 November 2016

### This version:

<http://wg21.link/P0516r0>

### Author:

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

### Audience:

SG1, LEWG, LWG

### Toggle Diffs:

☐ Hide deleted text

### Project:

ISO JTC1/SC22/WG21: Programming Language C++

---

## Table of Contents

- 1 GB 62
- 2 SG1 and LEWG Response to GB 62
- 3 LWG Feedback
- 4 Proposed Wording
  - 4.1 Changes to `shared_future`
  - 4.2 Changes to `future`
  - 4.3 Acknowledgements
- References
  - Informative References

This paper proposes a resolution to C++17 CD comment GB 62, which addresses operations on `shared_future` which have wide contracts.

### § 1. GB 62

There is an implicit precondition on most `shared_future` operations that `valid() == true`, 30.6.7p3. The list of exempted functions seems copied directly from class `future`, and would also include copy operations for `shared_futures`, which are copyable. Similarly, this would be a wide contract that cannot throw, so those members would be marked `noexcept`.

### § 2. SG1 and LEWG Response to GB 62

SG1 agreed with the proposed wording in the NB comment for `shared_future`. They felt the move constructor should also be added for the list in `future` as well. There was unanimous consent to forward this paper to LEWG for design review. LEWG reviewed it and sent it to LWG.

### § 3. LWG Feedback

LWG made the following changes:

- Rejected the addition of the constructors from the pre-condition whitelist (as the precondition has no meaning before the object is constructed).
- Added a note clarifying that it is valid to copy from a future for which `valid()` is false.

There was unanimous consent in LWG to move this paper at the Issaquah 2016 Friday plenary.

## § 4. Proposed Wording

The proposed changes are relative to [\[N4604\]](#), the Committee Draft for C++17.

### § 4.1. Changes to `shared_future`

Apply the following changes to 30.6.7 **[futures.shared\_future]** paragraph 3:

The effect of calling any member function other than the destructor, the move-assignment operator, the copy-assignment operator, or `valid` on a `shared_future` object for which `valid()` is false is undefined. [ *Note: It is valid to copy or move from a `shared_future` object for which `valid()` is false — end note* ] [ *Note: Implementations are encouraged to detect this case and throw an object of type `future_error` with an error condition of `future_errc::no_state`. — end note* ]

Apply the following changes to the class synopsis for `shared_future` in 30.6.7 **[futures.shared\_future]**:

```
namespace std {
    template <class R>
    class shared_future {
    public:
        shared_future() noexcept;
        shared_future(const shared_future& rhs) noexcept;
        shared_future(future<R>&&) noexcept;
        shared_future(shared_future&& rhs) noexcept;
        ~shared_future();
        shared_future& operator=(const shared_future& rhs) noexcept;
        shared_future& operator=(shared_future&& rhs) noexcept;

        // retrieving the value
        see below get() const;

        // functions to check state
        bool valid() const noexcept;

        void wait() const;

        template <class Rep, class Period>
            future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;

        template <class Clock, class Duration>
            future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
    };
}
```

Apply the following changes to the definition of `shared_future`'s copy constructor in 30.6.7 **[futures.shared\_future]**:

```
shared_future(const shared_future& rhs) noexcept;
```

*Effects:* Constructs a `shared_future` object that refers to the same shared state as `rhs` (if any).

*Postcondition:* `valid()` returns the same value as `rhs.valid()`.

Apply the following changes to the definition of `shared_future`'s copy assignment operator in 30.6.7 [**futures.shared\_future**]:

```
shared_future& operator=(const shared_future& rhs) noexcept;
```

*Effects:*

- Releases any shared state (30.6.4);
- assigns the contents of `rhs` to `*this`. [ *Note:* As a result, `*this` refers to the same shared state as `rhs` (if any). — *end note* ]

*Postconditions:* `valid() == rhs.valid()`.

## § 4.2. Changes to `future`

Apply the following changes to 30.6.6 [**futures.unique\_future**] paragraph 3:

The effect of calling any member function other than the destructor, the move-assignment operator, or `valid` on a `future` object for which `valid()` is `false` is undefined. [ *Note:* It is valid to move from a `future` object for which `valid() == false` — *end note* ]

## § 4.3. Acknowledgements

Thanks to Jonathan Wakely for assisting with the wording.

## § References

### § Informative References

#### [N4604]

Richard Smith. C++17 CD Ballot Document, 12 July 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4604.pdf>